

Assignment 1

Mapper Class:-

```
package org.ajinkya;
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;
public class WC_Mapper extends MapReduceBase implements
Mapper<LongWritable,Text,Text,IntWritable>{
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
    public void map(LongWritable key, Text value,OutputCollector<Text,IntWritable> output,
        Reporter reporter) throws IOException{
        String line = value.toString();
        StringTokenizer tokenizer = new StringTokenizer(line);
        while (tokenizer.hasMoreTokens()){
            word.set(tokenizer.nextToken());
            output.collect(word, one);
        }
    }
}
```

Reducer Class:-

```
package org.ajinkya;

import java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;

public class WC_Reducer extends MapReduceBase implements
Reducer<Text,IntWritable,Text,IntWritable> {
    public void reduce(Text key, Iterator<IntWritable>
values,OutputCollector<Text,IntWritable> output,
        Reporter reporter) throws IOException {
        int sum=0;
        while (values.hasNext()) {
            sum+=values.next().get();
        }
    }
}
```

```

        output.collect(key,new IntWritable(sum));
    }
}

```

Runner Class:-

```

package org.ajinkya;
import java.io.IOException;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.mapred.TextInputFormat;
import org.apache.hadoop.mapred.TextOutputFormat;
public class WC_Runner {
    public static void main(String[] args) throws IOException{
        JobConf conf = new JobConf(WC_Runner.class);
        conf.setJobName("WordCount");
        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);
        conf.setMapperClass(WC_Mapper.class);
        conf.setCombinerClass(WC_Reducer.class);
        conf.setReducerClass(WC_Reducer.class);
        conf.setInputFormat(TextInputFormat.class);
        conf.setOutputFormat(TextOutputFormat.class);
        FileInputFormat.setInputPaths(conf,new Path(args[0]));
        FileOutputFormat.setOutputPath(conf,new Path(args[1]));
        JobClient.runJob(conf);
    }
}

```

Input [File:-](#)

The cat sat on the mat. The cat looked around the room. It was a quiet room, and the cat liked the quiet

Output [File:-](#)

```

It      1
The     2
a       1
and     1
around  1
cat     3
liked   1
looked  1
mat.    1
on      1
quiet   2
room,   1

```

room. 1
sat 1
the 4
was 1

Assignment 3

Mapper Class:-

```
package org.sales;

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.*;
public class SaleCountryMapper extends MapReduceBase implements
Mapper<LongWritable, Text,
    Text, IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable>
        output, Reporter reporter) throws IOException {
        String valueString = value.toString();
        String[] SingleCountryData = valueString.split("-");
        output.collect(new Text(SingleCountryData[0]), one);
    }
}
```

Reducer Class:-

```
package org.sales;

import java.io.IOException;
import java.util.*;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.*;
public class SaleCountryReducer extends MapReduceBase implements Reducer<Text,
    IntWritable, Text, IntWritable> {
    public void reduce(Text t_key, Iterator<IntWritable> values,
        OutputCollector<Text,IntWritable> output, Reporter reporter) throws
IOException {
        Text key = t_key;
        int frequencyForCountry = 0;
        while (values.hasNext()) {
// replace type of value with the actual type of our value
            IntWritable value = (IntWritable) values.next();
            frequencyForCountry += value.get();
        }
        output.collect(key, new IntWritable(frequencyForCountry));
    }
}
```

Runner Class:-

```
package org.sales;
```

```
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapred.*;
public class SaleCountryRunner {
    public static void main(String[] args) {
        JobClient my_client = new JobClient();
// Create a configuration object for the job
        JobConf job_conf = new JobConf(SaleCountryRunner.class);
        // Set a name of the Job
        job_conf.setJobName("SalePerCountry");
// Specify data type of output key and value
        job_conf.setOutputKeyClass(Text.class);
        job_conf.setOutputValueClass(IntWritable.class);
// Specify names of Mapper and Reducer Class
        job_conf.setMapperClass(org.sales.SaleCountryMapper.class);
        job_conf.setReducerClass(org.sales.SaleCountryReducer.class);
// Specify formats of the data type of Input and output
        job_conf.setInputFormat(TextInputFormat.class);
        job_conf.setOutputFormat(TextOutputFormat.class);
// Set input and output directories using command line arguments,
//arg[0] = name of input directory on HDFS, and arg[1] = name of output directory
        //to be created to store the output file.
        FileInputFormat.setInputPaths(job_conf, new Path(args[0]));
        FileOutputFormat.setOutputPath(job_conf, new Path(args[1]));
        my_client.setConf(job_conf);
        try {
// Run the job
            JobClient.runJob(job_conf);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Input [File:-](#)

Pune
Mumbai
Nashik
Pune
Nashik
Kolapur

Output [File:-](#)

Kolapur	1	
Mumbai	1	
Nashik		2
Pune	2	